

Technical Requirements Document

ApplyWizz Workday Dry-Run Automation Engine V1

1. Product Name

ApplyWizz Workday Dry-Run Automation Engine

2. Technical Goal

The goal is to build a controlled Workday automation system using a future-ready architecture.

The product should first work as an internal operator dashboard, but the codebase should be ready to become a mobile application later.

Because of this, the frontend should be built using **React Native with Expo**, not plain React or only Next.js.

The system will allow ApplyWizz operators to:

- create candidate profiles
 - upload one resume
 - connect one Zoho email account
 - paste Workday job links
 - start dry-run automation
 - review extracted Workday questions
 - approve risky answers
 - stop before final submission
 - submit only after human approval
-

3. Core Technical Decision

We will build the frontend using:

``text id="ndngsc" Expo React Native + Expo Router + React Native Web

This gives us:

- web dashboard now
- mobile app later

- shared UI components
- shared business logic
- future Android/iOS app support

The backend, database, and automation worker will remain separate.

4. High-Level Architecture

```text id="8v9owq"

React Native / Expo Dashboard

↓

Supabase Auth

↓

Supabase PostgreSQL

↓

Supabase Storage

↓

Node.js Playwright Worker

↓

Workday Website

↓

HTML Capture

↓

ScrapeGraphAI

↓

OpenAI Answer Mapping

↓

Manual Review / Dry Run

## 5. Frontend

### 5.1 Framework

Use:

```text id="w6p0mm" Expo React Native

5.2 Routing

Use:

```
```text id="9a7z3d"
Expo Router
```

## 5.3 Web Support

Use:

```
```text id="6axagj" React Native Web
```

This allows the same React Native codebase to run as a web dashboard.

5.4 Language

Use:

```
```text id="kgucoc"
TypeScript
```

All frontend code must be written in TypeScript.

## 5.5 UI Styling

Use:

```
```text id="aw2r85" NativeWind
```

NativeWind gives Tailwind-like styling for React Native.

5.6 UI Component Direction

The UI should be:

- clean
- modern
- dark-mode friendly
- operator-focused
- responsive for desktop web
- ready for mobile later

Primary web dashboard should work well on:

- MacBook
- desktop monitor
- tablet

Mobile app can come later.

6. Backend

6.1 Backend Runtime

Use:

```
```text id="wtj3qp"
Node.js
+
TypeScript
```

## 6.2 API Layer

Use:

```
```text id="8axj5d" Supabase client + Node.js worker APIs
```

The frontend should communicate mainly with Supabase for:

- authentication
- candidate data
- job links
- run statuses
- manual review data
- logs
- screenshots

The Playwright worker should communicate with Supabase using the service role key.

6.3 Backend Responsibilities

The backend should handle:

- candidate data
- resume metadata
- job queue
- application run status
- extracted questions
- mapped answers
- manual review items

- screenshots
- logs
- OTP logs
- Workday account metadata

7. Database

7.1 Database Provider

Use:

```
```text id="2h0xo7"
Supabase PostgreSQL
```

## 7.2 Database Type

Use:

```
```text id="dhzpgd" PostgreSQL
```

7.3 Required Tables

The database should include:

```
```text id="zcp6fs"
users
candidates
candidate_resumes
candidate_answers
question_bank
workday_accounts
zoho_mailboxes
job_links
application_runs
run_steps
extracted_questions
mapped_answers
manual_review_items
otp_logs
screenshots
automation_logs
```

## 7.4 Database Rules

The database must:

- use UUID primary keys
  - use created\_at and updated\_at timestamps
  - use Row Level Security where appropriate
  - keep service-role operations only inside backend/worker
  - not expose private candidate data unnecessarily
  - not expose credentials to frontend
  - store sensitive tokens encrypted
- 

## 8. Authentication

### 8.1 Auth Provider

Use:

```text id="eoqq1j" Supabase Auth

8.2 Login Method for V1

Use:

```text id="n2pm9b"  
Email + Password

### 8.3 Future Login Options

Can add later:

- Google OAuth
- role-based permissions
- admin/operator roles
- audit logs

### 8.4 User Roles

V1 should support at least:

```text id="8jdpag" admin operator viewer

8.5 Role Permissions

Admin:

- manage users
- manage settings
- view all candidates
- approve final submit

Operator:

- create candidates
- upload resumes
- add job links
- run dry-runs
- review questions

Viewer:

- view dashboards and reports only

9. Storage

9.1 Storage Provider

Use:

```
```text id="h9m3x9"
Supabase Storage
```

## 9.2 Required Buckets

Create private buckets:

```
```text id="vsbg1f" candidate-resumes run-screenshots playwright-traces html-captures
```

9.3 Storage Rules

All buckets must be private.

The frontend should access files only through signed URLs.

The worker can upload files using the Supabase service role key.

10. Browser Automation

10.1 Automation Engine

Use:

```text id="vq8kn8"  
Playwright

## 10.2 Runtime

Use:

```text id="tqjbox" Node.js Playwright Worker

10.3 Worker Responsibilities

The Playwright worker should:

- pick queued jobs from Supabase
- open Workday job link
- detect page state
- login or create account
- read OTP when needed
- upload resume
- reach questionnaire page
- capture HTML
- capture screenshots
- fill safe answers
- detect conditional questions
- stop before final submit
- write all logs to Supabase

10.4 Worker Hosting

Use one of:

```text id="12nvms"  
Railway  
Render  
Fly.io  
VPS



For V1, a simple VPS or Railway worker is acceptable.

Do not run Playwright inside Vercel serverless functions.

---

## 11. Question Extraction

### 11.1 Extraction Tool

Use:

```
```text id="xbc1z0" ScrapeGraphAI
```

11.2 Purpose

ScrapeGraphAI should extract structured questions from Workday HTML.

It should return:

- question label
- field type
- options
- required or optional status
- section name
- page step
- visible/hidden status
- confidence score

11.3 HTML Processing

Before sending HTML to ScrapeGraphAI, the worker should:

- remove scripts
- remove styles
- remove unrelated layout HTML
- keep form labels
- keep input names
- keep aria labels
- keep select options
- keep radio options
- keep checkbox labels
- keep visible text

12. AI Answer Mapping

12.1 AI Provider

Use:

```
```text id="2pyxlz"
OpenAI
```

## 12.2 Purpose

OpenAI should be used only for:

- semantic question matching
- classifying question risk
- mapping question to approved answer
- explaining why manual review is needed

OpenAI should not invent answers.

## 12.3 Mapping Rule

The answer mapping flow should be:

```
```text id="gjx5iz"
Extracted Workday Question ↓ Normalize Question ↓ Check Candidate Answer Bank ↓
Check Global Question Bank ↓ Use OpenAI for semantic match ↓ Return confidence score ↓ Auto-fill only
if safe ↓ Manual review if risky or unknown
```

13. Email OTP

13.1 Email Provider

Use:

```
```text id="p7j69h"
Zoho Mail API
```

## 13.2 Purpose

Zoho Mail API should be used to:

- detect Workday OTP email
- read OTP
- save OTP usage log

- continue account creation or login

### 13.3 OTP Rules

The system should:

- search only OTP-related emails
  - avoid reading unrelated emails
  - timeout if OTP is not found
  - never expose full email inbox to frontend
  - log OTP success or failure
- 

## 14. Hosting and Deployment

### 14.1 Frontend Hosting

Use:

```text id="v3dah8" Expo Web hosted on Vercel

The React Native Web dashboard can be deployed to Vercel.

14.2 Database Hosting

Use:

```text id="v4i5hi"  
Supabase

Supabase will host:

- PostgreSQL database
- Auth
- Storage

### 14.3 Worker Hosting

Use:

```text id="h6ipdz" Railway / Render / Fly.io / VPS

Preferred V1 choice:

```
```text id="tuxifi"
Railway or VPS
```

Reason:

Playwright needs a long-running environment with browser support.

## 14.4 Why Not Vercel for Worker

Do not run the Playwright worker on Vercel because:

- browser automation is heavy
  - serverless timeout issues
  - background queue processing is not ideal
  - Playwright browsers need stable runtime
- 

## 15. Third-Party APIs

### 15.1 Supabase

Purpose:

- database
- authentication
- storage
- signed URLs

Tier:

- paid recommended for production
- 

### 15.2 OpenAI

Purpose:

- question matching
- risk classification
- answer mapping

Tier:

- paid API required
-

### 15.3 ScrapeGraphAI

Purpose:

- Workday HTML question extraction

Tier:

- paid or self-hosted depending on final setup
- 

### 15.4 Zoho Mail API

Purpose:

- OTP email reading
- candidate mailbox verification

Tier:

- depends on Zoho account plan
- 

### 15.5 Vercel

Purpose:

- host Expo web dashboard

Tier:

- free for testing
  - paid recommended for production
- 

### 15.6 Railway / Render / Fly.io / VPS

Purpose:

- host Playwright worker

Tier:

- paid recommended
-

## 16. Key Libraries

### 16.1 Frontend Libraries

Use:

```
```text id="4ll7kv" expo expo-router react-native react-native-web nativewind @supabase/supabase-js react-hook-form zod zustand lucide-react-native
```

16.2 Backend / Worker Libraries

Use:

```
```text id="f52idu"
playwright
@supabase/supabase-js
zod
dotenv
openai
axios
cheerio
pino
```

### 16.3 Testing Libraries

Use:

```
```text id="f2e3xq" vitest typescript eslint prettier
```

Optional later:

```
```text id="5j3iy9"
playwright/test
```

---

## 17. Folder Structure

Recommended monorepo structure:

```
```text id="pz7cfd" applywizz-workday-automation/ | ├── apps/ | ├── mobile-web/ | | ├── app/
| | | ├── (auth)/ | | | ├── (dashboard)/ | | | ├── candidates/ | | | ├── jobs/ | | |
| | | ├── runs/ | | | ├── manual-review/ | | | ├── settings/ | | | | | ├── components/ | |
| | | ├── hooks/ | | | ├── lib/ | | | ├── stores/ | | | ├── styles/ | | | ├── app.json | | | └──
```

```

package.json | | | └── worker/ | ├── src/ | | | ├── jobs/ | | | ├── playwright/ | | | ├──
workday/ | | | ├── zoho/ | | | ├── scrapegraph/ | | | ├── openai/ | | | ├── supabase/ | | |
└── logs/ | | | └── index.ts | └── package.json | ├── packages/ | ├── shared/ | | | ├──
types/ | | | ├── schemas/ | | | ├── constants/ | | | └── utils/ | | | └── database/ | ├──
migrations/ | ├── seed/ | └── types/ | ├── docs/ | ├── 01-prd.md | ├── 02-trd.md |
└── 03-app-flow.md | ├── 04-ui-ux.md | ├── 05-backend-schema.md | └── 06-
implementation-plan.md | ├── .env.example | ├── package.json | ├── pnpm-workspace.yaml | └──
README.md

```

```

---

## 18. Naming Conventions

### 18.1 Files

Use kebab-case for files:

```text id="gzltye"
candidate-card.tsx
job-link-table.tsx
manual-review-item.tsx

```

## 18.2 Components

Use PascalCase for components:

```
```text id="pfw7n2" CandidateCard JobLinkTable ManualReviewItem
```

```

### 18.3 Database Tables

Use snake_case:

```text id="ep35z4"
candidate_resumes
application_runs
manual_review_items

```

## 18.4 Variables

Use camelCase in TypeScript:

```
```text id="f18wd3" candidateId jobLinkId applicationRunId
```

18.5 Environment Variables

Use uppercase snake case:

```
```text id="bwqqfh"
SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY
OPENAI_API_KEY
```

---

## 19. Environment Variables

### 19.1 Frontend Environment Variables

```
```text id="mq0c40" EXPO_PUBLIC_SUPABASE_URL EXPO_PUBLIC_SUPABASE_ANON_KEY
EXPO_PUBLIC_APP_ENV
```

19.2 Worker Environment Variables

```
```text id="d04cap"
SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY
SUPABASE_STORAGE_BUCKET_RESUMES
SUPABASE_STORAGE_BUCKET_SCREENSHOTS
SUPABASE_STORAGE_BUCKET_TRACES
SUPABASE_STORAGE_BUCKET_HTML_CAPTURES

OPENAI_API_KEY
OPENAI_MODEL

SCRAPEGRAPHAI_API_KEY
SCRAPEGRAPHAI_ENDPOINT

ZOHIO_CLIENT_ID
ZOHIO_CLIENT_SECRET
ZOHIO_REDIRECT_URI

WORKER_CONCURRENCY
WORKER_POLL_INTERVAL_MS
WORKER_MAX_RETRIES

ENCRYPTION_KEY
```



```
APP_ENV
LOG_LEVEL
```

### 19.3 Local Development Variables

```
```text id="wakukq" DATABASE_URL DIRECT_URL NODE_ENV
```

```
---

## 20. Technical Constraints

### 20.1 Must Use

The system must use:

```text id="5jnz3h"
Expo React Native
Supabase
Playwright
ScrapeGraphAI
OpenAI
Zoho Mail API
TypeScript
```

### 20.2 Must Not Use in V1

The system must not use:

```
```text id="9ioq11" Browser Use Agent Ship SkillClaw AgentHub Alibaba Page Agent invisible_playwright
Obscura Firecrawl Crawl4AI
```

```
These are optional future tools only.

### 20.3 Must Not Do

The system must not:

- auto-submit without human approval in V1
- guess risky answers
- bypass CAPTCHA
- use anti-detect browser tooling
- expose service role key to frontend
- expose candidate credentials in UI
- read unrelated Zoho emails
```

- modify candidate resume automatically
- apply using multiple resumes in V1

21. Security Requirements

The system must:

- store resumes in private Supabase Storage bucket
- store screenshots in private bucket
- use signed URLs for private files
- encrypt Workday credentials
- encrypt Zoho tokens
- use service role key only in worker
- use anon key only in frontend
- enable RLS on user-facing tables
- log sensitive actions
- avoid unnecessary PII exposure
- separate frontend and worker permissions

22. Data Privacy Requirements

Candidate data is sensitive.

The system should:

- collect only necessary candidate data
- avoid storing unnecessary mailbox content
- avoid exposing raw OTP emails
- avoid sending full resume content to AI unless needed
- send only necessary question and answer data to OpenAI
- retain logs only as long as needed
- allow deletion of candidate data later

23. Risk Classification Logic

Every extracted question should be classified into one of these risk levels:

```text id="gktm8w"

low

medium

high

unknown

## **Low Risk**

Can be auto-filled if answer exists.

Examples:

- name
- email
- phone
- LinkedIn
- GitHub
- portfolio
- education
- work history

## **Medium Risk**

Can be auto-filled only with high confidence and approved answer.

Examples:

- availability
- notice period
- relocation
- expected salary
- preferred location

## **High Risk**

Must go to manual review.

Examples:

- visa sponsorship
- work authorization
- disability
- veteran status
- gender
- race
- ethnicity
- criminal history
- background check consent
- legal certification

## **Unknown**

Must go to manual review.

## 24. Worker State Machine

The Playwright worker should follow this state machine:

```
""text id="6m914k" queued ↓ starting ↓ opening_job_link ↓ detecting_page_state ↓
login_or_create_account ↓ waiting_for_otp ↓ otp_verified ↓ uploading_resume ↓ resume_uploaded ↓
reaching_questionnaire ↓ capturing_html ↓ extracting_questions ↓ mapping_answers ↓
filling_safe_answers ↓ checking_conditional_questions ↓ manual_review_required OR dry_run_complete ↓
human_approval_required ↓ submitted OR stopped
```

---

### ## 25. Error Handling Requirements

The worker must handle:

- invalid job link
- Workday page timeout
- apply button not found
- account already exists
- login failed
- OTP not found
- OTP expired
- resume upload failed
- questionnaire not reached
- extraction failed
- mapping failed
- unsupported field type
- duplicate application
- final submit disabled
- manual review required

For every error, save:

- current URL
- step name
- screenshot
- error message
- timestamp
- retry count
- suggested reason

---

### ## 26. Observability

The system must save:

- application run status
- run steps
- screenshots
- Playwright traces
- extracted HTML
- extracted questions
- mapped answers
- errors
- retries
- timestamps

The dashboard should show enough information for a non-technical operator to understand what happened.

Developers should be able to debug using:

- logs
- screenshots
- traces
- HTML captures

---

## ## 27. Testing Requirements

### ### 27.1 Unit Tests

Use Vitest for:

- question normalization
- risk classification
- answer mapping rules
- status transitions
- URL validation

### ### 27.2 Integration Tests

Test:

- Supabase connection
- resume upload
- job link creation
- worker queue pickup
- OTP parser
- ScrapeGraphAI response parser
- OpenAI mapping response parser

### ### 27.3 Browser Tests

Use Playwright tests later for:

- opening Workday page
- detecting login page
- detecting upload field
- extracting questionnaire page

### ### 27.4 Manual Testing

Test at least:

```
```text id="zhopuz"
5 Workday tenants for early testing
20 Workday tenants before internal rollout
50 Workday tenants before production confidence
```

28. Deployment Requirements

28.1 Environments

Use three environments:

```
```text id="rdpdlr" local staging production
```

### ### 28.2 Deployment Flow

```
```text id="kdtitl"
Local development
↓
Staging Supabase
↓
Staging worker
↓
Test Workday links
↓
Production deploy
```

28.3 Production Rule

Production worker must start in dry-run mode only.

Auto-submit should be disabled by default.

29. Development Process

Use Loop Engineering style:

``text id="4zhk20" Plan ↓ Implement ↓ Test ↓ Review ↓ Fix ↓ Verify ↓ Move to next phase

Each phase should have:

- implementation task
- tests
- verification command
- screenshot or output proof
- git status
- human approval before next phase

30. Recommended Build Phases

Phase 1: Project Setup

Build:

- monorepo
- Expo app
- worker app
- shared package
- TypeScript config
- linting
- formatting

Phase 2: Supabase Setup

Build:

- database migrations
- storage buckets
- RLS policies
- seed data

Phase 3: Auth

Build:

- Supabase Auth
- login screen
- protected routes
- operator session

Phase 4: Candidate Management

Build:

- candidate list
- create candidate
- edit candidate
- candidate detail

Phase 5: Resume Upload

Build:

- upload resume
- store in private bucket
- link to candidate

Phase 6: Job Link Input

Build:

- add one Workday link
- bulk paste links
- duplicate detection
- job queue status

Phase 7: Worker Foundation

Build:

- Supabase queue polling
- run creation
- step logging
- screenshot upload

Phase 8: Workday Open

Build:

- open job link
- detect page state
- save screenshot

Phase 9: Login / Account Creation

Build:

- login flow
- account creation flow
- credential storage

Phase 10: Zoho OTP

Build:

- Zoho API connection
- OTP search
- OTP extraction
- OTP logging

Phase 11: Resume Upload to Workday

Build:

- upload file to Workday
- verify upload success

Phase 12: Question Extraction

Build:

- HTML capture
- HTML compression
- ScrapeGraphAI extraction
- question storage

Phase 13: Answer Mapping

Build:

- candidate answer bank
- global question bank
- OpenAI semantic mapping
- risk classification

Phase 14: Form Filling

Build:

- text input filling

- dropdown filling
- radio filling
- checkbox filling
- safe-answer gate

Phase 15: Conditional Questions

Build:

- rescan after answers
- diff questions
- fill newly safe questions
- send unknowns to review

Phase 16: Manual Review

Build:

- review queue
- approve answer
- edit answer
- save to bank
- continue run

Phase 17: Dry-Run Summary

Build:

- final summary
- readiness score
- stop before submit
- human approval screen

Phase 18: Controlled Submit

Build:

- submit only after approval
- save submission proof
- final status

31. Future Technical Roadmap

V2

Add:

- Workday tenant profiles
- better retry rules
- Playwright trace viewer
- safe auto-submit for low-risk jobs
- Hermes `/learn` for repair suggestions

V3

Add:

- Greenhouse
- Lever
- iCIMS
- analytics dashboard
- candidate application reports

V4

Add:

- private SkillClaw
- Agent Ship for multiple internal agents
- Alibaba Page Agent for dashboard copilot

32. Final Technical Recommendation

Build the system with:

```text id="2jx7h4"

Expo React Native

+

React Native Web

+

Supabase

+

Node.js Playwright Worker

+

ScrapeGraphAI

+

OpenAI

+

Zoho Mail API

This gives ApplyWizz:

- web dashboard now
- mobile app later
- strong database foundation
- safe automation
- human review
- scalable future architecture

The first version should stay focused on:

```
text id="u3j61o"
One candidate
+
One resume
+
One Zoho email
+
One Workday link
+
Question extraction
+
Manual review
+
Dry-run before submit
```

Do not add extra agent frameworks in V1.

The most important technical goal is to prove that the system can reliably reach the Workday questionnaire page and extract questions correctly.